



Housing Dataset EDA

A Coding Club Mentor's Guide

Welcome! You've just done something every data scientist does before touching a machine learning model: **Exploratory Data Analysis (EDA)**. Your notebook loads a housing dataset (`Task3.csv` — 545 houses, 13 columns, this looks like the classic Kaggle "Housing Prices" dataset, where prices are typically listed in Indian Rupees ₹) and systematically gets to know it — first with tables and numbers, then with charts.

This guide walks through every single code cell and every single chart in your notebook, explains what it does, why it matters, and what it's actually telling you about the housing market in this data. Let's dig in.



SECTION 1: Step-by-Step Code Explanation

Step 1 — Importing the Toolkit

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

What it does: This loads three libraries into your notebook and gives them short nicknames (`pd` , `plt` , `sns`) so you don't have to type the full names every time.

- **pandas** → handles tabular data (think: Excel spreadsheets, but programmable).
- **matplotlib.pyplot** → the foundational plotting engine for Python.
- **seaborn** → sits on top of matplotlib and makes statistical charts (heatmaps, box plots, regression lines) much easier to build and prettier by default.

Why this step matters: Every EDA notebook starts here. Without these imports, none of your later `pd.read_csv()` , `sns.heatmap()` , or `plt.show()` calls would work. It's the equivalent of laying out your tools before starting a project.

Step 2 — Loading the Dataset

```
df = pd.read_csv('Task3.csv')
```

What it does: `pd.read_csv()` reads a CSV (Comma-Separated Values) file from disk and converts it into a DataFrame — pandas' core data structure, essentially a table with rows and labeled columns. It's stored in the variable `df`, which is now the "master copy" of your dataset for the rest of the notebook.

Why this step matters: You can't explore what you haven't loaded. This is the single most important line in the notebook — every subsequent cell operates on `df`.

Basic Data Exploration

(This matches the markdown header you added in the notebook — nice touch, by the way! Labeling sections like this is exactly how professional notebooks are organized.)

Step 3 — Peeking at the Top Rows

```
display(df.head())
```

What it does: `df.head()` shows the first 5 rows by default. `display()` is a Jupyter-specific function that renders it as a clean HTML table instead of plain text.

Why this step matters: This is your "sanity check." Did the file load correctly? Do the column names look right? Are the values in a sensible format (numbers where you expect numbers, yes/no where you expect categories)? You confirmed here that `df` has columns like price, area, bedrooms, and several yes/no amenity columns.

Step 4 — Peeking at a Random Sample

```
display(df.sample(5))
```

What it does: `df.sample(5)` pulls 5 random rows instead of the first 5.

Why this step matters: `.head()` only shows you the beginning of the file — if the data was sorted or entered in some pattern, the first few rows could be misleading. `.sample()` gives you a more honest, unbiased snapshot of what the whole dataset looks like.

Step 5 — Peeking at the Bottom Rows

```
display(df.tail())
```

What it does: `df.tail()` shows the last 5 rows.

Why this step matters: This checks that the file loaded completely and didn't get cut off, and that there's no junk data (like summary rows or footnotes) accidentally appended at the end of the CSV.

Step 6 — Structural Overview

```
df.info()
```

What it does: Prints a compact summary: total rows (545), total columns (13), each column's name, how many non-null values it has, and its data type (`int64` for whole numbers, `object` for text/categories).

Why this step matters: This is one of the most important diagnostic commands in all of pandas. In one glance, you learned:

- The dataset has 545 rows and 13 columns, no more, no less.
- Every column has 545 non-null values — meaning there's **no missing data** to clean up.
- 6 columns are numeric (price, area, bedrooms, bathrooms, stories, parking) and 7 are categorical text (mainroad, guestroom, basement, hotwaterheating, airconditioning, prefarea, furnishingstatus).

That numeric/categorical split matters a lot for everything that comes next — it determines which columns you can average or correlate, and which you need to group or count instead.

Step 7 — Explicit Missing Value Check

```
display(df.isnull().sum())
```

What it does: `df.isnull()` returns a same-shaped table of True/False values (True wherever a cell is missing). `.sum()` then adds those up per column, turning True/False into counts.

Why this step matters: `df.info()` already hinted at this, but this line makes it explicit and impossible to miss: the result was 0 for every single column. This dataset is squeaky clean — no imputation, no dropping rows, no missing-value strategy needed. That's a great, low-friction dataset to practice EDA on.

Step 8 — Confirming Data Types

```
display(df.dtypes)
```

What it does: Lists each column's data type on its own — a more focused version of what `df.info()` already showed.

Why this step matters: This is a deliberate double-check before doing any numeric analysis (like the correlation heatmap coming up). It confirms price, area, bedrooms, bathrooms, stories, and parking are all `int64` (true numbers you can do math on), while the rest are `object` (text categories like "yes"/"no" or "furnished") that need different treatment — grouping, counting, or encoding — rather than averaging directly.

Step 9 — Statistical Summary

```
display(df.describe())
```

What it does: `.describe()` runs standard summary statistics — count, mean, standard deviation, min, the 25th/50th/75th percentiles (quartiles), and max — but only on the numeric columns.

Why this step matters: This is where numbers start telling a story. Some highlights from your output:

Column	Mean	Min	Median (50%)	Max
price	₹47.7 lakh	₹17.5 lakh	₹43.4 lakh	₹1.33 crore
area (sq ft)	5,151	1,650	4,600	16,200
bedrooms	2.97	1	3	6
bathrooms	1.29	1	1	4
stories	1.81	1	2	4
parking	0.69	0	0	3

Notice that the mean price (₹47.7 lakh) is higher than the median price (₹43.4 lakh). That gap is a classic sign of a **right-skewed distribution** — a few expensive houses (up to ₹1.33 crore) are pulling the average upward, while most houses cluster at more modest prices. Keep that in mind — it shows up again in the charts below.

Step 10 — Correlation Heatmap

```
numerical_cols = df.select_dtypes(include=['int64', 'float64']).columns
corr_matrix = df[numerical_cols].corr()

plt.figure(figsize=(12, 10))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt='.2f', linewidths=.5)
plt.title('Correlation Matrix of Numerical Features')
plt.show()
```

What it does, line by line:

- `df.select_dtypes(include=['int64', 'float64'])` grabs only the numeric columns — you can't calculate a correlation between numbers and text.
- `.corr()` computes the Pearson correlation coefficient between every pair of numeric columns, producing a grid where each cell ranges from -1 to +1.
- `sns.heatmap(...)` draws that grid as a color-coded square, with `annot=True` printing the actual number in each cell and `cmap='coolwarm'` mapping high values to warm (red) colors and low values to cool (blue) colors.

Why this step matters: This is arguably the single most important chart in any EDA notebook. It instantly tells you which features are worth investigating further as potential drivers of your target variable (price), without having to eyeball dozens of individual scatter plots first. It's your "where should I look next?" compass.

Step 11 — Binning Area and Counting Houses

```
import numpy as np
df['area_bins'] = pd.cut(df['area'], bins=5)
house_counts = df['area_bins'].value_counts().sort_index()

plt.figure(figsize=(10, 6))
sns.barplot(x=house_counts.index.astype(str), y=house_counts.values,
palette='viridis')
plt.xlabel('Area Intervals')
plt.ylabel('Number of Houses')
plt.title('Number of Houses per Area Interval')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()

display(house_counts)
```

What it does:

- `pd.cut(df['area'], bins=5)` takes the continuous area column and slices its full range into 5 equal-width intervals (a technique called binning or discretization), storing which bin each house falls into as a new column, `area_bins`.
- `.value_counts()` counts how many houses fall into each bin; `.sort_index()` puts the bins in order from smallest to largest instead of by frequency.
- The `sns.barplot()` block then visualizes those counts.

Why this step matters: Binning turns a continuous number (area, which could be any value from 1,650 to 16,200) into a small number of categories you can easily count and compare — this is a univariate distribution check. It answers: "what does the spread of house sizes in this dataset actually look like?" before you start relating area to anything else.

Exploring Factors Affecting House Prices

(Your second markdown header — great signal that you're moving from "getting to know the data" to "answering specific questions about price.")

Step 12 — Price vs. Area Scatter Plot

```
plt.figure(figsize=(10, 6))
sns.scatterplot(x='area', y='price', data=df)
sns.regplot(x='area', y='price', data=df, scatter=False, color='red',
line_kws={'linestyle': '--'})
plt.title('House Price vs. Area')
plt.xlabel('Area (sq ft)')
plt.ylabel('Price')
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()
```

What it does: `sns.scatterplot()` plots every single house as one dot, positioned by its area (x-axis) and price (y-axis). `sns.regplot(..., scatter=False)` then overlays just a fitted straight regression line (the dashed red line) on top, without re-plotting the dots — showing the best-fit linear trend through the cloud of points, along with a shaded confidence band around it.

Why this step matters: This is a bivariate analysis — it examines the relationship between exactly two variables. It's the natural follow-up to seeing area show up with the highest correlation to price in the heatmap: now you're visually confirming that relationship instead of just trusting a single number. You already captured this well in your own markdown note — larger houses do tend to command higher prices, and the trend is clearly upward-sloping.

Step 13 — Furnishing Status Pie Chart

```
furnishing_counts = df['furnishingstatus'].value_counts()

plt.figure(figsize=(8, 8))
plt.pie(furnishing_counts, labels=furnishing_counts.index, autopct='%1.1f%%',
startangle=140)
plt.title('Distribution of Houses by Furnishing Status')
plt.axis('equal')
plt.show()
```

What it does: `.value_counts()` counts how many houses fall into each of the three furnishingstatus categories. `plt.pie()` then draws each category as a wedge sized proportionally to its count, with `autopct='%1.1f%%'` automatically labeling each wedge with its percentage share.

Why this step matters: This is a categorical distribution check — the counterpart to the numeric `describe()` you ran earlier, but for a text column. Before you can meaningfully compare prices across furnishing categories (which is exactly what the next two cells do), you need to know how many houses are in each group and whether any group is too small to trust.

Step 14 — Average Price by Furnishing Status (Table)

```
average_price_by_furnishing = df.groupby('furnishingstatus')
['price'].mean().reset_index()
display(average_price_by_furnishing)
```

What it does: `.groupby('furnishingstatus')` splits the dataset into three buckets (one per furnishing category), `['price'].mean()` calculates the average price within each bucket, and `.reset_index()` turns the result back into a tidy, flat table instead of a grouped object.

Why this step matters: This is the "split-apply-combine" pattern — one of the most powerful and commonly used techniques in all of data analysis. It directly answers a business question: "Does how a house is furnished actually relate to how much it sells for?" — and produces the exact numbers the next chart will visualize.

Step 15 — Average Price by Furnishing Status (Bar Chart)

```
plt.figure(figsize=(10, 6))
sns.barplot(x='furnishingstatus', y='price', data=average_price_by_furnishing,
            palette='viridis')
plt.title('Average House Price by Furnishing Status')
plt.xlabel('Furnishing Status')
plt.ylabel('Average Price')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```

What it does: Takes the small summary table from Step 14 and turns it into a bar chart — one bar per furnishing category, with height representing average price.

Why this step matters: Humans read bar heights far faster than they read a table of numbers. This turns your `groupby` result into an instantly digestible visual — you can tell at a glance which category commands the highest average price without comparing digits row by row.

Step 16 — Price Distribution by Bedroom Count (Box Plot)

```
plt.figure(figsize=(10, 6))
sns.boxplot(x='bedrooms', y='price', data=df, palette='coolwarm')
plt.title('House Price Distribution by Number of Bedrooms')
plt.xlabel('Number of Bedrooms')
plt.ylabel('Price')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```

What it does: For each unique value of `bedrooms` (1 through 6), `sns.boxplot()` draws a box summarizing the spread of prices within that group: the line inside the box is the median, the box edges are the 25th and 75th percentiles (the "middle 50%" of prices), the whiskers extend to the typical range, and any individual dots beyond the whiskers are flagged as outliers.

Why this step matters: Unlike the bar chart in Step 15 (which only shows one number — the average — per group), a box plot shows the entire distribution, including spread and outliers, for each group. This is a much richer way to compare price across a categorical-like numeric variable (bedrooms) than a single average would be, and it's exactly the right chart choice when you suspect a group might contain unusually priced outliers (which, as you'll see below, it does).



SECTION 2: Visualizations Summary & Key Insights

Below is a visual breakdown chart by chart, summarizing exactly what the EDA teaches us about the dataset.

1. Correlation Heatmap

What it shows: A 6×6 grid comparing every numeric column against every other numeric column.

How to read it: Each cell's value ranges from -1 to +1. +1 = perfect positive relationship (as one goes up, the other always goes up); -1 = perfect negative relationship (as one goes up, the other always goes down); 0 = no linear relationship at all. The diagonal is always 1.00 because every column is perfectly correlated with itself. Color reinforces the number — in this coolwarm palette, redder cells are higher correlations, bluer cells are lower.

What it found:

Feature	Correlation with Price	Strength
area	0.54	Moderate positive
bathrooms	0.52	Moderate positive
stories	0.42	Weak-to-moderate positive
parking	0.38	Weak positive
bedrooms	0.37	Weak positive

2. Number of Houses per Area Interval (Bar Chart)

What it shows: 545 houses sliced into 5 equal-width area ranges, with bar height = number of houses in that range.

How to read it: Taller bars mean that size range is more common in the dataset.

What it found: Heavily right-skewed. About 88% of all houses (476 of 545) fall in the two smallest bins (1,635–7,470 sq ft), while only 2 houses in the entire dataset exceed 13,290 sq ft.

Area Range (sq ft)	Number of Houses	Share
1,635 – 4,560	272	49.9%
4,560 – 7,470	204	37.4%
7,470 – 10,380	54	9.9%
10,380 – 13,290	13	2.4%
13,290 – 16,200	2	0.4%

3. House Price vs. Area (Scatter + Regression Line)

What it shows: Every house as a dot positioned by area (x) and price (y), with a red dashed line showing the overall linear trend.

How to read it: An upward-sloping line = positive relationship. How tightly the dots hug the line tells you how strong and reliable that relationship is.

What it found: A clear upward trend confirms area's 0.54 correlation from the heatmap. But look closely — the dots fan out wider as area increases (this is called *heteroscedasticity*). Small houses (under ~4,000 sq ft) are tightly priced together near the bottom, but mid-size houses (6,000–9,000 sq ft) show huge price variation — some sell for ₹30 lakh, others for over ₹1.3 crore at similar sizes. Area alone doesn't fully explain price — other features (bathrooms, amenities, location) are clearly doing a lot of the remaining work.

4. Distribution of Houses by Furnishing Status (Pie Chart)

What it shows: The 545 houses split by furnishing category.

How to read it: Wedge size (and the labeled percentage) = share of the total.

What it found: Semi-furnished is the most common status (41.7%), followed by unfurnished (32.7%), with fully furnished homes the smallest group (25.7%). All three groups are large enough (well over 100 houses each) that comparing their average prices, as the next chart does, is statistically meaningful.

5. Average House Price by Furnishing Status (Bar Chart)

What it shows: One bar per furnishing category, height = average price within that category.

What it found: A clean, ordered relationship:

Furnishing Status	Average Price
Furnished	₹54.96 lakh
Semi-furnished	₹49.08 lakh
Unfurnished	₹40.14 lakh

Fully furnished homes sell for ~37% more on average than unfurnished ones — one of the clearest, most actionable patterns in the whole notebook.

6. House Price Distribution by Number of Bedrooms (Box Plot)

What it shows: Price spread for each bedroom count from 1 to 6.

How to read it: Higher boxes = higher typical prices for that group; taller boxes = more price variability within that group; dots above the whiskers = outlier houses priced unusually high for their bedroom count.

What it found: Median price rises steadily from 1 bedroom (~₹27 lakh) up through 5 bedrooms (~₹56 lakh) — but then dips at 6 bedrooms (~₹47 lakh). This is very likely a small-sample artifact rather than a real market effect — the area-bins chart already told us there are very few large/unusual houses in this dataset, so the 6-bedroom group probably contains only a handful of houses, making its "average" unreliable. Also notice: the most extreme high-price outliers (up to ₹1.33 crore) appear in the 3- and 4-bedroom groups, not among the highest bedroom counts — reinforcing that bedroom count alone isn't the main price driver either.



Top Real-Estate Insights

Strongest price drivers (among numeric features):

- **Area (0.54)** — the single strongest numeric predictor of price.
- **Bathrooms (0.52)** — nearly as strong as area, and often overlooked by beginners.
- **Stories (0.42)** — multi-story homes trend pricier.

Weaker (but still positive) relationships:

Parking spaces (0.38) and bedrooms (0.37) are the weakest of the six numeric features — they matter, but far less than area or bathroom count. Notably, no numeric feature in this dataset is negatively correlated with price, and none reaches "strong" territory (above 0.7) — a good reminder that no single number fully explains a house's price.

An important limitation worth flagging: the heatmap only ever looked at the 6 numeric columns.

Categorical amenities like airconditioning, mainroad, hotwaterheating, and prefarea were never tested for correlation with price at all — `.corr()` simply can't process "yes"/"no" text. Your furnishing-status analysis (Steps 13–15) got around this nicely by using `groupby()`, and the same technique could be applied to those other categorical columns as a great next experiment.

Distribution & outlier patterns across the notebook:

- **Price is right-skewed** (`describe()`) showed mean ₹47.7L > median ₹43.4L: most homes are moderately priced, with a handful of luxury outliers (up to ₹1.33 crore) pulling the average up.
- **Area is also right-skewed**: ~88% of houses are under 7,470 sq ft; ultra-large properties (13,000+ sq ft) are rare (0.4% of the dataset).
- **Price variance increases with area** (heteroscedasticity in the scatter plot) — meaning predicting price gets harder, not easier, for larger homes.
- **The 6-bedroom price dip** is a small-sample effect, not a genuine market trend — a good beginner lesson in not over-trusting thin slices of data.

Actionable real-estate takeaways:

- 🛠️ **Renovations**: If you had to renovate one thing to boost resale value, adding a bathroom looks like a stronger lever than adding a bedroom, given its higher correlation with price.
- 🛋️ **Staging**: Furnishing status is a genuinely persuasive selling point — the ~37% average price gap between furnished and unfurnished homes suggests staging/furnishing a home before selling could be worth the investment.
- 📏 **Pricing**: Area matters, but it's a starting point, not the full story — two similarly sized houses in this dataset can differ in price by millions, so amenities and bathroom count deserve equal attention in any pricing model.
- ⚠️ **Caution**: Be cautious drawing conclusions about 6-bedroom homes (and any other thin slice of this data) — small groups produce noisy, unreliable averages.

📌 Bonus: Why This Notebook Is a Great Portfolio Piece

Since you're building out your Data Science stack for your GitHub portfolio, this notebook already hits the core EDA checklist recruiters and interviewers look for: data validation (`.info()` , `.isnull().sum()`), summary statistics (`.describe()`), a correlation study, and multiple well-chosen chart types (heatmap, scatter+regression, pie, grouped bar, box plot) — each picked deliberately for the type of question it answers. A natural next step, if you want to extend it, would be a simple linear regression model using area and bathrooms as predictors — a clean bridge from EDA into your modeling practice.

Great work — keep this exploratory rigor going into every dataset you touch! 🎉